

Ton idée est solide et répond à un vrai problème opérationnel : beaucoup de bailleurs gèrent encore loyers, charges et états des locataires avec des cahiers, Excel ou WhatsApp.

Le point important est que ton projet n'est pas simplement une "application de location", mais plutôt un **ERP léger de gestion immobilière locative**.

Tu as déjà identifié les trois piliers essentiels :

- gestion des propriétés
- gestion des locataires
- gestion financière (loyers + charges)

L'idée d'un dashboard admin global est pertinente, mais il faut bien structurer les rôles et responsabilités dès le départ.

1. Analyse fonctionnelle du projet

Acteurs du système

1. Super Admin

C'est toi ou l'entreprise qui gère la plateforme.

Responsabilités :

- créer/suspendre des comptes bailleurs
- voir les statistiques globales
- gérer abonnements/paiements
- support technique
- monitoring

Donc oui, un dashboard admin est une bonne idée.

2. Bailleur (propriétaire)

Fonctions principales :

- gérer ses propriétés
- ajouter des maisons/appartements/chambres
- ajouter des locataires
- enregistrer les paiements
- suivre les impayés
- consulter les charges

- recevoir des alertes
- générer des reçus

C'est le cœur du système.

3. Locataire (optionnel mais fortement recommandé)

Fonctions :

- consulter son solde
- voir historique des paiements
- télécharger reçus
- voir factures eau/électricité
- recevoir rappels

Même si tu ne fais pas cette partie au début, prévois l'architecture pour l'ajouter plus tard.

2. Modules du système

A. Gestion des propriétés

Entités :

- Propriété
- Bâtiment
- Appartement/Chambre
- Statut (occupé/libre)

Exemple :

Immeuble A
├─ Chambre 1
├─ Chambre 2
└─ Studio 3

B. Gestion des locataires

Données :

- nom
- téléphone
- CNI

- contrat
 - date d'entrée
 - caution
 - loyer mensuel
-

C. Gestion des paiements

Très important.

Tu dois gérer :

- loyers
- avances
- arriérés
- paiements partiels
- historique
- pénalités

Exemple :

Mois	Montant	Statut
Avril	50 000	payé
Mai	50 000	impayé

D. Gestion des charges

Tu as déjà identifié le besoin :

- eau
- électricité
- wifi
- gardiennage
- nettoyage

Deux approches possibles :

Charges fixes

Exemple :

- wifi = 5 000/mois

Charges variables

Exemple :

- eau selon compteur
- électricité selon consommation

Prévois les deux dès l'architecture.

E. Notifications

Très important pour la valeur produit.

Canaux :

- SMS
- WhatsApp
- Email
- Push mobile

Exemples :

- “Votre loyer expire dans 3 jours”
 - “Facture d'eau non payée”
 - “2 mois d'impayés”
-

F. Reporting / Analytics

Le bailleur doit voir :

- revenus mensuels
 - taux d'occupation
 - total impayés
 - propriétés rentables
 - locataires à risque
-

3. Architecture recommandée

Option recommandée : SaaS Multi-tenant

Tu héberges une seule plateforme.

Chaque bailleur possède :

- ses propriétés
- ses locataires
- ses données isolées

C'est la meilleure architecture pour :

- évoluer
 - monétiser
 - maintenance simple
-

4. Technologies recommandées

FRONTEND

Web Dashboard

Je recommande fortement :

- React
- Next.js
- TypeScript

Pourquoi :

- moderne
- scalable
- excellent pour dashboards
- SEO si nécessaire
- énorme écosystème

UI :

- TailwindCSS
- shadcn/ui

Très bon combo pour dashboards professionnels.

MOBILE

Deux options :

Option 1 — Flutter (fortement recommandé)

Très bon pour :

- Android
- iPhone
- performance
- UI moderne

Excellent choix pour l'Afrique.

Option 2 — React Native

Possible si tu veux mutualiser avec React.

Mais Flutter est souvent plus propre pour apps métier.

5. Backend recommandé

Meilleur choix : NestJS

Stack :

- Node.js
- NestJS
- TypeScript

Pourquoi NestJS :

- architecture entreprise
- propre
- scalable
- auth robuste
- modules clairs
- parfait pour SaaS

Exemple modules :

- AuthModule
- PropertyModule
- TenantModule
- BillingModule

- NotificationModule
-

6. Base de données

PostgreSQL

Très bon choix.

Pourquoi :

- robuste
- transactions fiables
- relations complexes
- parfait pour finance/location

Tu vas avoir beaucoup de relations :

- bailleur → propriétés
- propriété → chambres
- chambre → locataire
- locataire → paiements

PostgreSQL est idéal.

7. ORM recommandé

Prisma

Excellent avec TypeScript.

Avantages :

- migrations propres
 - typage fort
 - rapide à développer
 - maintenable
-

8. Authentification

Je recommande :

- JWT
- Refresh tokens
- RBAC (Role Based Access Control)

Rôles :

- SUPER_ADMIN
 - BAILLEUR
 - LOCATAIRE
-

9. Paiements mobiles (très important en Afrique)

Tu dois penser très tôt à :

- Mobile Money MTN
- Orange Money
- éventuellement Stripe

Au Cameroun, c'est un énorme avantage produit.

10. Infrastructure / Déploiement

Hébergement recommandé

Backend

- Docker
- Railway
- Render
- VPS Hetzner
- AWS plus tard

Base de données

- Supabase PostgreSQL
ou
 - Neon
-

11. Sécurité

Très important car données financières.

Prévoir :

- hash passwords (bcrypt)
 - audit logs
 - sauvegardes
 - permissions strictes
 - HTTPS
 - rate limiting
-

12. Architecture technique idéale

Architecture moderne recommandée

```
Frontend Web (Next.js)
|
Mobile App (Flutter)
|
API Gateway
|
NestJS Backend
|
PostgreSQL + Prisma
|
Notification Services
SMS / WhatsApp / Email
```

13. Fonctionnalités MVP (version 1)

Ne fais pas tout immédiatement.

MVP recommandé

Phase 1

- Auth
- Gestion propriétés
- Gestion locataires
- Paiements loyers

- Tableau de bord
- Historique
- Reçus PDF

Phase 2

- Charges eau/électricité
- Notifications SMS
- Statistiques avancées

Phase 3

- Paiement Mobile Money
 - Application mobile
 - Signature contrat
 - IA/prédiction impayés
-

14. Ce qu'il faut éviter

Mauvaise idée :

Créer :

- frontend compliqué
- microservices trop tôt
- architecture Kubernetes dès le début

Tu n'en as pas besoin.

15. Recommandation finale de stack

Stack que je recommande fortement

Frontend Web

- Next.js
- TypeScript
- Tailwind
- shadcn/ui

Mobile

- Flutter

Backend

- NestJS
- Prisma

Database

- PostgreSQL

Auth

- JWT + RBAC

Infra

- Docker
 - Railway/Render
 - Supabase
-

16. Vision business intéressante

Ton application peut évoluer vers :

- gestion immobilière complète
- comptabilité bailleurs
- agence immobilière
- paiement automatisé
- scoring locataire
- gestion maintenance
- marketplace immobilière

Le marché africain est encore peu structuré sur ces outils.

17. Conclusion

Le projet est :

- techniquement pertinent
- commercialement viable

- scalable
- très bon pour un SaaS

Et oui :

le dashboard admin global est une bonne idée, à condition de bien séparer :

- Super Admin plateforme
- Dashboard Bailleur
- éventuellement espace Locataire

Le point clé du succès sera :

1. simplicité d'utilisation
2. notifications automatiques
3. Mobile Money
4. gestion claire des impayés
5. UX adaptée aux bailleurs non techniques.